

CHECKLIST DE REPRODUCIBILIDAD DEL PIPELINE

Análisis de sentimiento en redes sociales aplicado al Mundial FIFA 2022

Autor: Juan Diego Ruiz Valverde — UNIR 2026

1. Información del entorno de ejecución

Todas las versiones registradas a continuación corresponden al entorno Conda TFM utilizado para ejecutar el pipeline.

Componente	Versión / Valor
Sistema operativo	Windows 10/11 (64-bit)
Python	3.10 (entorno Conda: TFM)
pandas	3.0.3
scikit-learn	1.8.0
spacy	3.8.14
transformers	5.8.1
sentence-transformers	5.5.1
torch	2.12.0
streamlit	1.58.0
Modelo spaCy (ES)	es_core_news_sm
Semilla global (seed)	42
Split train/test	80% / 20%
Modelo anotador principal	pysentimiento (RoBERTa ES)
Modelo anotador secundario	cardiffnlp/twitter-xlm-roberta-base-sentiment
Repositorio GitHub	github.com/jdruizv-web/sentiment-analysis-worldcup2022
DOI corpus Zenodo	https://doi.org/10.5281/zenodo.20726879
Checksum corpus canónico (SHA-256)	e97da11f850e569f7e326cff61ed63ad9b9f9fa0dea74d5edcf634bf0fe12bb9

Componente	Versión / Valor
Kappa inter-anotador (κ)	0.6356 (acuerdo sustancial)

2. Pasos para reproducir el pipeline

Paso 0 — Requisitos previos

1. Instalar Git: <https://git-scm.com/download/win>
2. Instalar Anaconda o Miniconda: <https://www.anaconda.com>
3. Clonar el repositorio:

```
git clone https://github.com/jdruizv-web/sentiment-analysis-worldcup2022.git
```

```
cd sentiment-analysis-worldcup2022
```
4. Crear y activar el entorno Conda:

```
conda create -n TFM python=3.10
```

```
conda activate TFM
```
5. Instalar dependencias:

```
pip install -r requirements.txt
```
6. Instalar el modelo de spaCy para español:

```
python -m spacy download es_core_news_sm
```
7. Descargar el corpus procesado desde Zenodo (DOI: 10.5281/zenodo.20726879) y colocar los archivos en la carpeta data/.

Paso 1 — Ingestión del corpus (ingest.py)

Coloca el archivo CSV original en data/ antes de ejecutar:

```
python src/ingest.py
```

Artefactos generados:

- data/corpus_canonical_v1.jsonl
- data/checksum.txt

Verificación del checksum:

```
certutil -hashfile data\corpus_canonical_v1.jsonl SHA256
```

Valor esperado: e97da11f850e569f7e326cff61ed63ad9b9f9fa0dea74d5edcf634bf0fe12bb9

Paso 2 — Preprocesamiento y etiquetado (preprocess.py)

Asegúrate de que preprocess_config.yml tiene label_strategy: "both" antes de ejecutar:

```
python src/preprocess.py
```

Artefactos generados:

- data/corpus_processed_v1.jsonl
- data/corpus_labeled_v1.jsonl
- data/sample_processed.csv
- data/preprocess_log.txt
- data/kappa_result.txt — debe contener $\kappa = 0.6356$

Paso 3 — Modelos clásicos con TF-IDF (baselines.py)

```
python src/baselines.py
```

Artefactos generados:

- results/results_baselines.csv
- models/models_baselines/NaiveBayes.pkl
- models/models_baselines/LogisticRegression.pkl
- models/models_baselines/SVM.pkl
- logs/experiments_log.txt

Paso 4 — Modelos con embeddings (embeddings.py)

Este paso puede tardar entre 30 minutos y 2 horas según el hardware disponible:

```
python src/embeddings.py
```

Artefactos generados:

- results/results_embeddings.csv
- models/models_embeddings/LogisticRegression_BETO.pkl
- models/models_embeddings/SVM_BETO.pkl
- models/models_embeddings/LogisticRegression_ST.pkl
- models/models_embeddings/SVM_ST.pkl
- logs/experiments_embeddings_log.txt

Paso 5 — Dashboard interactivo (app.py)

```
streamlit run dashboard/app.py
```

El dashboard abre automáticamente en el navegador en <http://localhost:8501>

Requiere que los artefactos de los pasos 2, 3 y 4 existan en sus carpetas correspondientes.

3. Checklist de verificación final

Elemento	Verificado	Notas / Ubicación
corpus_canonical_v1.jsonl generado	✓	
Checksum SHA-256 verificado	✓	
corpus_processed_v1.jsonl generado	✓	
corpus_labeled_v1.jsonl generado	✓	
kappa_result.txt contiene $\kappa = 0.6356$	✓	
results_baselines.csv generado	✓	
models/models_baselines/ contiene 3 modelos .pkl	✓	

Elemento	Verificado	Notas / Ubicación
results_embeddings.csv generado	✓	
models/models_embeddings/ contiene 4 modelos .pkl	✓	
logs/experiments_log.txt generado	✓	
logs/experiments_embeddings_log.txt generado	✓	
Dashboard funcional en localhost:8501	✓	
Repositorio público en GitHub	✓	
Corpus publicado en Zenodo con DOI	✓	
requirements.txt incluido en el repositorio	✓	
README.md con instrucciones de reproducción	✓	
Anexo 1 - Registro de tratamiento (PDF)	✓	
Anexo 2 - DPIA (DOCX/PDF)	✓	
Anexo 3 - Checklist de reproducibilidad (DOCX)	✓	

4. Parámetros clave del pipeline

Todos los parámetros configurables se centralizan en src/preprocess_config.yml. Los valores utilizados en el TFM son:

Parámetro	Valor usado en TFM	Descripción
label_strategy	both	Usa pysentimiento + cardiffnlp y calcula Kappa
emoji_strategy	map	Convierte emojis a palabras sentimentales
hashtag_strategy	normalize	#Mundial → mundial
lemmatize	false	Tokens originales (recomendado para Transformers)
seed	42	Semilla global para reproducibilidad
sample_size	500	Tamaño de la muestra aleatoria en CSV
label_validation_sample	150	Muestra para validación de etiquetado

5. Resultados esperados al reproducir

Al ejecutar el pipeline completo con los parámetros documentados, se deben obtener los siguientes resultados:

Modelos clásicos (results_baselines.csv)

Modelo	Precision	Recall	F1
Naïve Bayes	0.72	0.70	0.66
Logistic Regression	0.74	0.74	0.73
SVM	0.73	0.73	0.73

Modelos con embeddings (results_embeddings.csv)

Modelo	Precision	Recall	F1
LogisticRegression BETO	0.72	0.72	0.72
SVM BETO	0.73	0.73	0.73
LogisticRegression ST	0.79	0.80	0.79
SVM ST	0.78	0.78	0.78